

SIMATS ENGINEERING

ASSESSMENT TOOL 2

K. Lokesh Chandra
192425358

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5

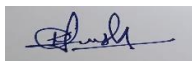
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I K. Lokesh Chandra(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Online Shopping Platform – Normalize to 3NF

The given relation is:

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

The question asks to identify functional dependencies and normalize the relation up to 3NF.

Functional Dependencies

The important functional dependencies are:

1. OrderID $\rightarrow$ CustID
2. CustID $\rightarrow$ CustName
3. ProdID $\rightarrow$ ProdName, Price
4. (OrderID, ProdID) $\rightarrow$ Qty

Therefore:

(OrderID, ProdID) $\rightarrow$ CustID, CustName, ProdName, Qty, Price

The candidate key is:

Candidate Key = (OrderID, ProdID)

1NF

The relation is in 1NF when every attribute contains atomic values.

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

2NF

There is a partial dependency:

- OrderID $\rightarrow$ CustID, CustName
- ProdID $\rightarrow$ ProdName, Price

These attributes depend only on part of the composite key (OrderID, ProdID).

Decompose into:

Customer

Customer(CustID, CustName)

Product

Product(ProdID, ProdName, Price)

Order

Order(OrderID, CustID)

OrderDetails

OrderDetails(OrderID, ProdID, Qty)

3NF

There is a transitive dependency:

CustID $\rightarrow$ CustName

Therefore customer information is separated into the Customer table.

Final 3NF design:

Table	Attributes	Key
Customer	CustID, CustName	CustID
Product	ProdID, ProdName, Price	ProdID
Order	OrderID, CustID	OrderID
OrderDetails	OrderID, ProdID, Qty	OrderID + ProdID

Justification

This decomposition removes **update, insertion and deletion anomalies**. Customer and product information are stored only once, reducing redundancy and improving data integrity.

2. Hospital Database – Normalize to BCNF

Given:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

The question requires identifying anomalies and normalizing to **BCNF**.

Functional Dependencies

Assuming each patient is assigned to one doctor and each ward has one name:

1. PID $\rightarrow$ PName, DocID, WardNo
2. DocID $\rightarrow$ DocName, Specialization
3. WardNo $\rightarrow$ WardName

The candidate key is:

PID

Anomalies

1. Update anomaly:

If a doctor's name or specialization changes, multiple patient records may have to be updated.

2. Insertion anomaly:

A doctor cannot be stored unless at least one patient is assigned to that doctor.

3. Deletion anomaly:

Deleting the last patient of a doctor may also remove information about that doctor.

BCNF Decomposition

Patient Table

Patient(PID, PName, DocID, WardNo)

Primary Key: **PID**

FD:

PID $\rightarrow$ PName, DocID, WardNo

Doctor Table

Doctor(DocID, DocName, Specialization)

Primary Key: **DocID**

FD:

$\text{DocID} \rightarrow \text{DocName}, \text{Specialization}$

Ward Table

Ward(WardNo, WardName)

Primary Key: **WardNo**

FD:

$\text{WardNo} \rightarrow \text{WardName}$

BCNF Verification

For every non-trivial FD, the determinant is a candidate key of its relation:

- $\text{PID} \rightarrow \text{PName}, \text{DocID}, \text{WardNo} \rightarrow \text{PID}$ is a key.
- $\text{DocID} \rightarrow \text{DocName}, \text{Specialization} \rightarrow \text{DocID}$ is a key.
- $\text{WardNo} \rightarrow \text{WardName} \rightarrow \text{WardNo}$ is a key.

Therefore, all three relations satisfy **BCNF**.

Conclusion

The decomposition eliminates redundancy and prevents insertion, deletion and update anomalies while maintaining data integrity.

3. University Enrollment – 2NF Decomposition

Given:

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

The question requires functional dependency analysis and **2NF decomposition**.

Functional Dependencies

1. $\text{SID} \rightarrow \text{SName}$
2. $\text{CourseID} \rightarrow \text{CourseName}, \text{Faculty}$
3. $(\text{SID}, \text{CourseID}) \rightarrow \text{Grade}$

The candidate key is:

(SID, CourseID)

Partial Dependencies

Since:

$\text{SID} \rightarrow \text{SName}$

and

$\text{CourseID} \rightarrow \text{CourseName}, \text{Faculty}$

these attributes depend on only part of the composite key.

Therefore, the original relation violates **2NF**.

2NF Decomposition

Student

Student(SID, SName)

Primary Key: **SID**

Course

Course(CourseID, CourseName, Faculty)

Primary Key: **CourseID**

Enrollment

Enrollment(SID, CourseID, Grade)

Primary Key:

(SID, CourseID)

Foreign Keys:

- SID → Student
- CourseID → Course

Final Structure

Table	Attributes	Primary Key
Student	SID, SName	SID
Course	CourseID, CourseName, Faculty	CourseID
Enrollment	SID, CourseID, Grade	SID + CourseID

Conclusion

The decomposition removes partial dependencies and ensures that every non-key attribute in Enrollment depends on the **whole composite key**.

4. Airline Booking – Normalize to BCNF

Given:

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

The question requires normalization up to **BCNF**.

Assumptions

Assume:

- Each passenger ID identifies exactly one passenger.
- Each flight number identifies one route.
- A flight on a particular date can have many passengers.
- A passenger has one seat on a particular flight/date.
- A seat number is unique within a particular flight/date.

Functional Dependencies

1. PassID → PassName
2. FlightNo → DeptCity, ArrCity
3. (FlightNo, Date, PassID) → Seat
4. (FlightNo, Date, Seat) → PassID

Candidate keys can therefore be:

- (FlightNo, Date, PassID)
- (FlightNo, Date, Seat)

BCNF Decomposition

Passenger

Passenger(PassID, PassName)

Primary Key: **PassID**

Flight

Flight(FlightNo, DeptCity, ArrCity)

Primary Key: **FlightNo**

Booking

Booking(FlightNo, Date, PassID, Seat)

Candidate Keys:

- (FlightNo, Date, PassID)
- (FlightNo, Date, Seat)

BCNF Verification

- PassID $\rightarrow$ PassName: PassID is the key of Passenger.
- FlightNo $\rightarrow$ DeptCity, ArrCity: FlightNo is the key of Flight.
- Booking dependencies have candidate keys as determinants.

Thus, the resulting relations satisfy **BCNF**.

Benefits

The decomposition avoids repeating:

- Passenger name for every booking.
- Departure and arrival cities for every flight.
- Passenger and flight information unnecessarily.

It also reduces update and deletion anomalies.

5. HR System – Normalize to 3NF

The assessment asks to analyze an HR system for a multinational company, identify functional and transitive dependencies, and normalize it to 3NF.

Assumed Initial Relation

Since the question does not provide a complete schema, assume:

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, CountryID, CountryName)

Functional Dependencies

1. EmpID $\rightarrow$ EmpName, DeptID, ManagerID, CountryID
2. DeptID $\rightarrow$ DeptName, ManagerID
3. ManagerID $\rightarrow$ ManagerName
4. CountryID $\rightarrow$ CountryName

Transitive Dependencies

There are transitive dependencies such as:

EmpID $\rightarrow$ DeptID $\rightarrow$ DeptName

and:

EmpID $\rightarrow$ ManagerID $\rightarrow$ ManagerName

and:

EmpID $\rightarrow$ CountryID $\rightarrow$ CountryName

These cause redundancy.

3NF Decomposition

Employee

Employee(EmpID, EmpName, DeptID, ManagerID, CountryID)

Primary Key: **EmpID**

Department

Department(DeptID, DeptName, ManagerID)

Primary Key: **DeptID**

Manager

Manager(ManagerID, ManagerName)

Primary Key: **ManagerID**

Country

Country(CountryID, CountryName)

Primary Key: **CountryID**

3NF Verification

Every non-key attribute depends on the key, the whole key, and nothing but the key.

The transitive dependencies have been removed.

Conclusion

The design reduces repeated department, manager and country information and prevents update, insertion and deletion anomalies.

6. Library Database – MVDs and 4NF

Given:

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

The question specifically asks to identify **multivalued dependencies (MVDs)** and decompose the relation to **4NF**.

Functional Dependencies

Assuming each member has one name and each book has fixed details:

1. MemberID $\rightarrow$ MemberName
2. BookID $\rightarrow$ Title, Author, Genre

Multivalued Dependency

A member can borrow multiple books independently.

Therefore:

MemberID $\twoheadrightarrow$ BookID

This means a member can be associated with multiple books independently of the book details.

Problem

The original table can create repeated combinations of:

- Member
- Book
- Borrow information

This results in redundancy and possible anomalies.

4NF Decomposition

Member

Member(MemberID, MemberName)

Primary Key: **MemberID**

Book

Book(BookID, Title, Author, Genre)

Primary Key: **BookID**

Borrow

Borrow(MemberID, BookID, BorrowDate)

Primary Key:

(MemberID, BookID, BorrowDate)

Foreign Keys:

- MemberID references Member.
- BookID references Book.

4NF Verification

A relation is in 4NF when every non-trivial MVD has a determinant that is a superkey.

The independent multivalued information has been separated into the Borrow relation.

Conclusion

The decomposition removes multivalued redundancy and allows members to borrow multiple books without unnecessarily duplicating member and book information.

7. Telecom Billing – Keys and Functional Dependencies

The question asks to identify **candidate keys, primary keys and functional dependencies before normalization.**

Assumed Telecom Schema

Assume:

Billing(BillID, CustomerID, CustomerName, PhoneNo, PlanID, PlanName, CallID, CallDate, Duration, Rate, Amount)

Functional Dependencies

1. BillID $\rightarrow$ CustomerID, PlanID
2. CustomerID $\rightarrow$ CustomerName, PhoneNo
3. PlanID $\rightarrow$ PlanName, Rate
4. CallID $\rightarrow$ CallDate, Duration
5. (CallID, Rate, Duration) $\rightarrow$ Amount

A more direct billing dependency can be represented as:

(CallID, Rate, Duration) $\rightarrow$ Amount

because:

Amount = Duration $\times$ Rate

Candidate Keys

Under the stated assumptions:

- **BillID** identifies a bill.
- **CallID** identifies an individual call record.

Therefore candidate keys are:

BillID, CallID

If the relation stores one row per call within a bill, a suitable composite key is:

(BillID, CallID)

Primary Key

A suitable primary key is:

BillID

for the bill-level relation.

For call details:

(BillID, CallID)

Foreign Keys

- CustomerID → Customer
- PlanID → Plan
- BillID → Bill

Summary

Attribute	Dependency
CustomerID	determines CustomerName, PhoneNo
PlanID	determines PlanName, Rate
BillID	determines customer and plan
CallID	determines call date and duration
Duration + Rate determines Amount	

Conclusion

Identifying these keys and dependencies before normalization helps detect partial and transitive dependencies and provides a proper basis for decomposition.

8. Retail Inventory – Complete Normalization

Given:

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

The question asks for complete normalization.

Functional Dependencies

Assume:

1. PID → PName, SupplierID, Category, Price, Warehouse
2. SupplierID → SupplierName

Therefore:

PID → SupplierID → SupplierName

This is a **transitive dependency**.

1NF

All attributes contain atomic values.

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

2NF

The assumed primary key is **PID**, which is a single attribute.

Therefore, there is no partial dependency and the relation satisfies 2NF.

3NF

There is a transitive dependency:

$PID \rightarrow SupplierID \rightarrow SupplierName$

So decompose into:

Product

Product(PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key: **PID**

Supplier

Supplier(SupplierID, SupplierName)

Primary Key: **SupplierID**

BCNF

In Product:

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

PID is the candidate key.

In Supplier:

$SupplierID \rightarrow SupplierName$

SupplierID is the candidate key.

Therefore both relations satisfy BCNF under the stated assumptions.

Final Normalized Design

Table	Attributes	Primary Key
Product	PID, PName, SupplierID, Category, Price, Warehouse	PID
Supplier	SupplierID, SupplierName	SupplierID

Benefits

- Supplier information is not repeated for every product.
- Updating supplier details requires only one row.
- Deletion of a product does not accidentally delete supplier information.
- Data integrity is improved.

9. School ERP – Fully Normalized Design up to BCNF

The assessment asks for a fully normalized School ERP design up to BCNF with justification, but the question does not provide the original poorly normalized schema.

Assumed Poorly Normalized Schema

Assume the School ERP initially stores:

SchoolERP(StudentID, StudentName, DeptID, DeptName, CourseID, CourseName, FacultyID, FacultyName, Grade)

Functional Dependencies

1. $StudentID \rightarrow StudentName, DeptID$
2. $DeptID \rightarrow DeptName$
3. $CourseID \rightarrow CourseName, FacultyID$
4. $FacultyID \rightarrow FacultyName$
5. $(StudentID, CourseID) \rightarrow Grade$

Candidate key:

(StudentID, CourseID)

Problems

There are partial dependencies:

$\text{StudentID} \rightarrow \text{StudentName}, \text{DeptID}$

and:

$\text{CourseID} \rightarrow \text{CourseName}, \text{FacultyID}$

There are also transitive dependencies:

$\text{StudentID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

and:

$\text{CourseID} \rightarrow \text{FacultyID} \rightarrow \text{FacultyName}$

BCNF Decomposition

Student

Student(StudentID, StudentName, DeptID)

Primary Key: **StudentID**

Department

Department(DeptID, DeptName)

Primary Key: **DeptID**

Course

Course(CourseID, CourseName, FacultyID)

Primary Key: **CourseID**

Faculty

Faculty(FacultyID, FacultyName)

Primary Key: **FacultyID**

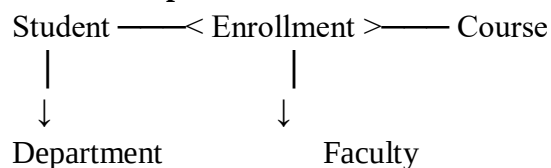
Enrollment

Enrollment(StudentID, CourseID, Grade)

Primary Key:

(StudentID, CourseID)

Relationship Structure



BCNF Justification

Each determinant is a candidate key in its respective relation:

- $\text{StudentID} \rightarrow \text{StudentName}, \text{DeptID}$
- $\text{DeptID} \rightarrow \text{DeptName}$
- $\text{CourseID} \rightarrow \text{CourseName}, \text{FacultyID}$
- $\text{FacultyID} \rightarrow \text{FacultyName}$
- $(\text{StudentID}, \text{CourseID}) \rightarrow \text{Grade}$

Therefore, the resulting design satisfies BCNF.

Advantages

The normalized design:

- Eliminates repeated student information.
- Eliminates repeated department information.
- Eliminates repeated faculty information.
- Prevents update anomalies.

- Maintains referential integrity.
- Makes the ERP easier to maintain and extend.

10. Movie Streaming Platform – 5NF

The question asks to identify **join dependencies**, decompose to **5NF**, and perform lossless-join verification.

Since the assessment does not specify the exact streaming schema, assume:

MovieStreaming(MovieID, ActorID, PlatformID)

Assume that the association between a movie, its actors and streaming platforms is independent.

Join Dependency

The relation may contain independent combinations:

- Movie $\leftrightarrow$ Actor
- Movie $\leftrightarrow$ Platform
- Actor $\leftrightarrow$ Platform

This can produce unnecessary combinations.

The join dependency can be represented as:

(MovieID, ActorID), (MovieID, PlatformID), (ActorID, PlatformID)

This means the original relation can be reconstructed by joining the three projections.

Decomposition

MovieActor

MovieActor(MovieID, ActorID)

Primary Key:

(MovieID, ActorID)

MoviePlatform

MoviePlatform(MovieID, PlatformID)

Primary Key:

(MovieID, PlatformID)

ActorPlatform

ActorPlatform(ActorID, PlatformID)

Primary Key:

(ActorID, PlatformID)

5NF Condition

A relation is in **5NF** when every non-trivial join dependency is implied by candidate keys.

The decomposition separates the independent many-to-many relationships.

Lossless-Join Verification

The original relation can be reconstructed as:

MovieStreaming

=

MovieActor

⋈ MoviePlatform

⋈ ActorPlatform

Therefore, joining the decomposed relations produces the original valid combinations under the stated independence assumption.

Example

Suppose:

MovieActor

MovieID ActorID

M1 A1

M1 A2

MoviePlatform

MovieID PlatformID

M1 P1

M1 P2

The decomposed relations preserve the independent relationships without storing the same movie-platform information repeatedly with every actor.

Conclusion

The 5NF decomposition minimizes redundancy caused by complex join dependencies and provides a lossless representation of the original relationship.